

Project Progress Report

Portable Windows Registry Forensic Analyzer

Live Windows Registry Analysis using Python winreg

Digital Forensics Internship - 2026 | National CERT (NCERT)

Project Area	Windows Artefacts and Registry Forensics
Tool Type	Portable live registry forensic analyzer
Main Language	Python
Main Windows API/Module	winreg
Current Interface	Console-based backend menu; GUI integration planned
Report Date	31 July 2026
Prepared By	Group 10 - Digital Forensics Internship

Confidentiality Note: This report avoids raw registry records, usernames, file names, and generated evidence outputs to protect system privacy.

1. Executive Summary

The project has been converted into a live Windows Registry forensic tool. Instead of relying mainly on offline hive files, the current scope is to read registry artefacts directly from the running Windows system using Python winreg. This approach is faster for demonstration, easier to convert into an EXE, and more suitable for a short internship timeline.

The core backend for Member 1 is functionally complete. The tool can collect system information, installed software details, startup programs, USB-related registry evidence, and current user activity artefacts. It can also generate JSON and CSV reports from a full scan.

The current version is console-based. It is ready for UI integration by the next member. The future UI can use Flask, HTML/CSS/JavaScript, or another Python-compatible interface, while calling the existing backend functions.

2. Current Status Snapshot

Area	Status	Progress	Remarks
Backend - Member 1	Working	90%	Core live registry modules and export modules are completed.
USB Testing	Partial	70%	Code is ready; needs final test on a system with USB history.
Reporting Export	Working	85%	JSON and multiple CSV outputs generated successfully.
EXE Preparation	Started	40%	Console EXE build flow started; final GUI packaging pending.
UI Development	Design stage	20%	Splash/dashboard mockups created; functional UI integration pending.

3. Project Objective and Scope

The purpose of this tool is to provide a simple, portable, and investigator-friendly way to collect useful Windows Registry forensic artefacts from a live Windows machine.

- Read live Windows Registry artefacts without manually opening Registry Editor.
- Collect key forensic information such as system details, installed software, auto-start programs, USB evidence, and user activity.
- Generate structured reports in JSON and CSV formats for later analysis, presentation, or documentation.
- Prepare the backend so a GUI dashboard can later call the same functions through buttons such as System Info, Software, USB Devices, Startup Programs, User Activity, and Generate Report.
- Build the tool as a portable Windows EXE so it can be run on a PC without manually installing the full development environment.

4. Current Architecture

Layer	Component	Purpose
User Interface	Console Menu (current), GUI planned	Allows the user to choose which artefact to scan.
Backend	Python modules	Contains separate modules for each registry artefact.
Windows API	winreg	Reads live registry keys and values from HKLM and HKCU.
Artefact Modules	system_info, installed_software, startup_programs, usb_devices, user_activity	Collect specific registry evidence in clean dictionary/list format.
Report Module	json_export, csv_export	Exports scan results into JSON and CSV files.

Portable Build	PyInstaller planned/started	Creates an EXE for easy execution on Windows PCs.
----------------	-----------------------------	---

5. Completed Backend Modules

Module	File	Main Output	Status
Common Registry Reader	backend/live_registry/common_reader.py	Reusable read functions for registry keys/values	Completed
System Information	backend/live_registry/system_info.py	Windows version, build, computer/user info, timezone	Working
Installed Software	backend/live_registry/installed_software.py	Software name, version, publisher, install location/date	Working
Startup Programs	backend/live_registry/startup_programs.py	Auto-start programs from HKLM/HKCU Run keys	Working/needs final review
USB Devices	backend/live_registry/usb_devices.py	USBSTOR and MountedDevices registry evidence	Code ready; USB test pending
User Activity	backend/live_registry/user_activity.py	RunMRU, TypedPaths, RecentDocs, UserAssist	Working
Report Export	backend/reports/json_export.py, csv_export.py	Full JSON report and separate CSV reports	Working

6. Testing Summary

Testing was performed on a Windows system using the console version of the tool. Sensitive raw results are not included in this report. The following summary confirms functional testing only.

Feature	Result	Status	Notes
System Information	Data collected	Pass	Windows details, system root, user profile, and timezone were shown.
Installed Software	29 programs detected	Pass	Collected from 64-bit, 32-bit, and current-user uninstall registry paths.
User Activity	RunMRU 4, TypedPaths 1, RecentDocs 275, UserAssist 88	Pass	Shows strong registry artefact extraction from HKCU.
Full Report Export	JSON + 8 CSV files generated	Pass	Full scan output saved successfully to output folder.
USB Devices	No USBSTOR found on office PC	Partial	Code is ready; needs testing on a PC with USB storage history and admin mode.

7. Forensic Artefacts Covered

Artefact	Registry Area	Forensic Value	Current Status
System Info	HKLM	Identifies OS version, system root, computer name, and timezone.	Working
Installed Software	HKLM/HKCU Uninstall	Shows software installed on the system and related registry view.	Working
Startup Programs	HKLM/HKCU Run	Helps identify auto-start/persistence entries.	Working
USB Evidence	HKLM USBSTOR / MountedDevices	Shows external storage or mounted device traces.	Code ready; test pending
User Activity	HKCU Explorer keys	Shows Run commands, typed paths, recent documents, and UserAssist execution traces.	Working

8. Issues and Challenges Faced

- Install Date shown as None for some applications:** Some applications do not store InstallDate in their uninstall registry entry. This is expected behavior and not a code error.

- **USBSTOR path not found on office PC:** The PC may not have USB storage history, policy may restrict it, or admin rights may be needed. Testing will be repeated at home with a USB device.
- **RecentDocs output contains noise:** RecentDocs values are binary data. The tool extracts readable filenames, but extra shortcut/noise data may appear. Cleaning logic has been added and can be improved further.
- **Privacy risk in reports:** JSON/CSV outputs contain real host/user evidence. The output folder is ignored in Git to prevent accidental upload.
- **EXE path confusion:** The EXE must be run from the correct dist folder or copied to Desktop. GUI packaging is still pending.

9. Privacy and Safety Controls

- The tool is designed for read-only registry access. It does not modify registry keys.
- Generated reports are saved locally inside the output folder.
- The output folder, sample hives, build folders, and cache files should remain in .gitignore.
- Real logs, screenshots, generated CSV/JSON reports, and registry hive files should not be pushed to GitHub.
- For best USB/system-level results, the tool should be run as Administrator during testing.

10. Handover for UI Member

The backend is ready for UI integration. The UI member can call these functions when dashboard buttons are clicked:

UI Button	Backend Function/Module	Expected UI Display
System Information	SystemInfoReader().get_system_info()	System/OS details cards and table
Installed Software	InstalledSoftwareReader().get_installed_software()	Software table with name, version, publisher
Startup Programs	StartupProgramsReader().get_startup_programs()	Auto-start entries table
USB Devices	USBDevicesReader().read_usb_devices()	USB/mounted devices table
User Activity	UserActivityReader().get_user_activity()	RunMRU, TypedPaths, RecentDocs, UserAssist sections
Generate Report	generate_full_report() or report export classes	Progress bar plus report saved message

11. Next Work Plan

No.	Task	Description	Priority
1	USB final testing	Test USB module on home PC with actual USB device and administrator mode.	High
2	UI integration	Connect backend modules with Flask or selected GUI dashboard buttons.	High
3	Progress bar/loading state	Show scanning progress when a user clicks each button.	Medium
4	Report buttons	Add export JSON/CSV button in dashboard.	High
5	EXE packaging	Build final portable EXE after UI is ready.	High
6	Final testing	Test on at least two Windows systems.	High
7	Final documentation	Prepare final report, screenshots, and demo presentation.	Medium

12. Current Code Structure

```
Windows_Registry_Forensics_Tool/
├── backend/
│   ├── live_registry/
│   │   ├── common_reader.py
│   │   ├── system_info.py
│   │   ├── installed_software.py
│   │   ├── startup_programs.py
│   │   ├── usb_devices.py
│   │   └── user_activity.py
│   └── reports/
│       ├── json_export.py
│       └── csv_export.py
├── output/          # local reports only; not pushed to GitHub
├── main.py
├── requirements.txt
└── .gitignore
```

13. Conclusion

The Member 1 backend work is completed at core level. The live registry modules are working and the report export functionality has been tested successfully. The project is now ready to move into UI integration and final testing.

The most important pending backend validation is USB evidence testing on a system with actual USB history. Apart from this, only polishing, UI connection, and final EXE packaging remain.

Overall, the project is on track and has reached a practical working stage for demonstration and further development.